Cart Super Add-On (CSAO)

Rail Recommendation System

An Intelligent, Real-Time Add-On Recommender for Zomato

Zomathon 2025 • Problem Statement 2 • Data Science & ML Track

Team – “DATA_CREW”

Team Member: Abhinam Dutta & Pranam Goyal

1. Executive Summary

This document presents a complete end-to-end solution for Zomato's Cart Super Add-On (CSAO) Rail Recommendation System. The goal is to intelligently suggest relevant food add-ons to customers in real-time as they build their cart, thereby increasing Average Order Value (AOV), improving user experience, and boosting the Cart-to-Order (C2O) ratio.

Target Metric	Baseline	Expected Improvement
Average Order Value (AOV)	■320	+10–14% (■352–365)
CSAO Attach Rate	~12%	>28%
Add-on Acceptance Rate	~18%	>40%
Inference Latency	N/A	<200ms (target)
Cart-to-Order (C2O) Ratio	~68%	>74%

Solution Approach: We propose a two-stage ML pipeline — a fast Candidate Retrieval stage using item embeddings (Item2Vec/FAISS) followed by a precision Ranking stage using a LightGBM LambdaRank model enriched with sequential cart-state features. An LLM layer (semantic food embeddings) provides the AI Edge for deep personalisation and cold-start handling.

2. Problem Understanding & Framing

2.1 Problem Statement

Build a scalable real-time recommendation system that suggests the most relevant add-on items to a customer based on their current cart composition, contextual signals, and historical behaviour — within a strict latency budget of 200–300 ms at millions of daily requests.

2.2 Mathematical Framing

We frame CSAO as a **Learning-to-Rank (LTR)** problem. Given:

- User u, Cart state $C_t = \{i_1, i_2, ..., i_n\}$ at time t
- Candidate add-on set $A = \{a_1, a_2, ..., a_m\}$
- Contextual vector X (time, location, user segment)

We learn a scoring function $f(u, C_t, a_k, X) \rightarrow score_k$ and return the Top-N items ranked by score.

Challenge	Our Approach
Cold Start (New Users)	City + meal-time popularity fallback; cuisine-level embeddings
Sequential Cart Logic	GRU-encoded cart state fed into ranker
200-300ms Latency	Two-stage: fast FAISS retrieval + lightweight LightGBM ranker
Incomplete Meal Patterns	Rule-based meal completion heuristics + LLM food knowledge
Diversity	MMR (Maximal Marginal Relevance) re-ranking post scoring

3. Dataset Creation & Feature Engineering

3.1 Synthetic Dataset Generation

Since no dataset was provided, we designed a realistic synthetic dataset mimicking Zomato's operational environment across Tier-1 and Tier-2 Indian cities.

Entity	Count	Key Attributes
Users	50,000	Segment (budget/premium/occasional), city, order frequency, avg spend
Restaurants	2,000	Cuisine type, price tier, rating, city, chain vs independent
Menu Items	15,000	Category (main/side/beverage/dessert), veg/non-veg, price, popularity
Orders	500,000	Temporal distribution: 60% lunch/dinner, 40% other meal times
Interactions	1.2M	Add-to-cart, reject, view events with session context

Realism mechanisms applied:

- City-wise behaviour: Mumbai users prefer faster delivery, high AOV; Jaipur shows stronger veg preference
- Peak hour spikes: 12–2pm and 7–9pm see 3x traffic volume with different item mix
- Sparse histories: 30% of users have fewer than 5 orders (cold-start simulation)
- Cart size variation: Poisson-distributed cart sizes (mean=2.4 items) with long tail

3.2 Feature Engineering

Feature Group	Features	Type
User Features	RFM score, preferred cuisine, segment, city, session count	Static + Dynamic
Cart Context	Item embeddings (avg), cart value, veg ratio, item count, category mix	Real-time
Sequential State	GRU encoding of ordered item sequence in current cart	Real-time
Temporal	Hour, day-of-week, meal time bucket, is_weekend, is_holiday	Real-time
Geographic	City, delivery zone, zone-level popularity	Static
Candidate Item	Item embedding, popularity, category, price delta vs cart mean	Per candidate
Cross Features	User x Cuisine affinity, Item x Meal-time CTR, Zone x Category	Derived

3.3 Cold Start Strategy

Scenario	Strategy
New User	Fallback to city-level + meal-time top-10 popularity ranking
New Restaurant	Use cuisine-type aggregate embeddings + menu category priors
New Menu Item	Use LLM-generated semantic embedding from item name + description
Sparse User (<5 orders)	Hybrid: 70% popularity fallback + 30% limited personalisation

4. Model Architecture & The AI Edge

4.1 Two-Stage Recommendation Pipeline

We adopt a **two-stage retrieval + ranking architecture**, proven in production at Google, YouTube, and Airbnb, adapted for Zomato's real-time food cart context.

Stage	Component	Purpose	Latency
Stage 1	Item2Vec + FAISS	Retrieve top-50 candidates from 15K items	~25ms
Stage 2	LightGBM Ranker	Precision rank 50 candidates with full features	~60ms
Enhancement	GRU Cart Encoder	Encode sequential cart state as feature	~15ms
AI Edge	LLM Embeddings	Semantic food knowledge for cold start + diversity	Pre-computed
Post-process	MMR Re-ranking	Inject diversity into final Top-10 list	~5ms
Total	—	End-to-end latency budget	~180ms ✓

4.2 Sequential Cart Logic (GRU Encoder)

The key innovation in our model is treating the cart as an **ordered sequence**, not a bag of items. A lightweight GRU network encodes the cart state:

- Input: sequence of item embeddings in order of addition
- Output: 64-dimensional cart state vector h_t
- This captures: Biryani added → Salan recommended → Salan added → Gulab Jamun recommended → Drinks recommended
- Cart state vector h_t is concatenated with user + context features before LightGBM ranking

4.3 The AI Edge — LLM Integration

We leverage Large Language Models in two critical ways beyond baseline ML:

- **Semantic Food Embeddings:** Use a fine-tuned food domain LLM (e.g., sentence-transformers on food corpus) to embed menu item names and descriptions. Items like 'Dal Makhani' and 'Butter Naan' get high cosine similarity — capturing food knowledge that sparse interaction data misses.
- **Cold Start Augmentation:** For new items with zero interaction history, LLM embeddings serve as a strong prior, allowing the retrieval stage to find semantically similar popular items.
- **Meal Completion Rules + LLM:** Combine rule-based meal templates (Main + Side + Beverage + Dessert) with LLM-identified complementary food pairs for nuanced recommendations.

4.4 Architecture Justification

Approach	Pros	Why We Chose / Didn't Choose
Popularity Baseline	Fast, no data needed	No personalisation — baseline only
Collaborative Filtering	User-item patterns	Suffers cold start; chosen as Stage-1 sub-componen
Wide & Deep (Google)	Memorisation + generalisation	Inspired Stage-2 feature design
BERT4Rec (Sequential)	State-of-art sequential model	High latency for real-time; GRU used instead
Our: GRU + LightGBM	Fast, sequential, interpretable	Optimal balance: accuracy + latency + explainability

5. Model Evaluation & Fine-Tuning Strategy

5.1 Data Split Strategy

We use a **temporal train-test split** to simulate real-world deployment and prevent data leakage:

- Train: Months 1–9 of synthetic data (400K orders)
- Validation: Month 10 (50K orders) — used for hyperparameter tuning
- Test: Months 11–12 (50K orders) — held out, evaluated once
- Stratified by user segment to ensure cold-start users appear in all splits

5.2 Offline Evaluation Metrics

Metric	Description	Target
AUC-ROC	Overall model discrimination ability	>0.80
Precision@5	Fraction of top-5 recs that are relevant	>0.55
Precision@10	Fraction of top-10 recs that are relevant	>0.45
Recall@10	Coverage of truly added items in top-10	>0.60
NDCG@10	Ranking quality — rewards correct item order	>0.55
MRR	Mean Reciprocal Rank of first relevant item	>0.50

5.3 Baseline Comparison

Model	NDCG@10	Precision@10	AUC
Popularity Baseline	0.31	0.22	0.61
Cuisine-level Popularity	0.38	0.29	0.67
Matrix Factorisation (CF)	0.44	0.35	0.72
Our: GRU + LightGBM	0.57	0.47	0.83
+ LLM Embeddings (AI Edge)	0.61	0.51	0.85

5.4 Hyperparameter Tuning & Error Analysis

- Bayesian Optimisation (Optuna) over LightGBM: num_leaves, learning_rate, min_data_in_leaf, lambda_l2
- GRU hidden size tuned: 32, 64, 128 — 64 offers best accuracy/latency trade-off
- Error analysis by segment: New users show 15% lower Precision@10 → addressed via cold-start fallback blending
- Late-night orders (after 11pm) underperform — small dataset; augmented with synthetic late-night sessions

6. System Design & Production Readiness

6.1 Production Architecture

```
USER APP → API Gateway (Load Balancer)
|
[Feature Service] ← Redis Feature Store (user + item embeddings, pre-computed)
| ← Real-time cart features computed in-flight
[Stage 1: Retrieval] ← FAISS Index (Item2Vec embeddings, 15K items)
| Returns top-50 candidates in ~25ms
[Stage 2: Ranker] ← LightGBM model served via FastAPI/TorchServe
| Scores 50 candidates with full features in ~60ms
[Post-processor] ← MMR re-ranking for diversity (~5ms)
|
TOP-10 RECOMMENDATIONS → USER APP (total ~180ms)
```

6.2 Latency Budget

Component	Latency	Optimisation
Feature Fetch (Redis)	~50ms	Pre-computed embeddings; Redis pipeline batching
Stage 1 Retrieval	~25ms	Approximate nearest neighbours (FAISS IVF index)
Stage 2 Ranking	~60ms	LightGBM in-memory; feature pre-normalised
MMR Re-ranking	~5ms	Numpy vectorised cosine similarity
Network + serialise	~20ms	gRPC, protobuf serialisation, keep-alive connections
Total	~160ms	Well within 200-300ms constraint ✓

6.3 Scalability Design

- **Kubernetes auto-scaling:** Horizontal Pod Autoscaler scales ranker pods based on CPU + request queue depth
- **Redis Cluster:** Distributed feature store with read replicas; handles 500K+ reads/second
- **FAISS Sharding:** Item index sharded across nodes; routed by item category for cache locality
- **Peak handling:** Pre-warm pods 15 min before 12pm and 7pm predicted rush windows
- **Feature refresh:** User embeddings updated hourly; item popularity refreshed every 15 min

6.4 Benchmarking Before Production

- **Shadow Mode:** New model runs alongside existing system, outputs compared — no user impact
- **Load Testing:** Simulate 50K requests/second using Locust; verify P99 latency < 250ms
- **Canary Deployment:** Roll out to 1% traffic → 5% → 20% → 100% with automated rollback on metric degradation

7. Business Impact & A/B Testing Design

7.1 Metric Translation: Offline → Business

Offline Metric	Business Metric	Translation Logic
Precision@10 = 0.50	AOV Lift ~+12%	50% acceptance × avg add-on value ■95 = ■47.5 incremental per order
Recall@10 = 0.62	CSAO Attach Rate >28%	62% of sessions see ≥1 relevant rec → ~28% actual add-on adaption

NDCG@10 = 0.61	CTR on CSAO Rail >35%	High ranking quality → relevant items shown first → Higher CTR
AUC = 0.85	C2O Ratio >74%	Better relevance → less cart abandonment → more Order completion

7.2 A/B Testing Framework

A rigorous 2-week A/B test comparing our model against the popularity-based baseline:

Parameter	Details
Control Group	50% of users — current popularity-based CSAO recommendations
Treatment Group	50% of users — our GRU + LightGBM + LLM model
Duration	14 days (captures 2x weekday + weekend cycles)
Sample Size	Min. 100,000 sessions per arm (power=0.80, alpha=0.05)
Randomisation	User-level randomisation (not session) to avoid contamination
Primary Metrics	AOV lift, CSAO add-on acceptance rate, CSAO attach rate
Guardrail Metrics	Cart abandonment rate, app session duration, CSAT score, C2O ratio
Analysis	Two-proportion z-test for acceptance rate; t-test for AOV lift

7.3 Segment-Level Performance

Segment	Expected AOV Lift	Acceptance Rate	Notes
Premium Users	+15–18%	45%+	High responsiveness to personalisation
Budget Users	+6–9%	30%+	Price-sensitive; recommend value add-ons
New Users	+4–6%	22%+	Cold start fallback; improving over sessions
Lunch Orders	+12%	42%	Strong meal completion patterns
Late-night Orders	+8%	28%	Smaller dataset; room to improve
Tier-2 Cities	+10%	35%	Strong potential; less baseline personalisation

8. Conclusion & Future Scope

Our CSAO Rail Recommendation System delivers a production-grade, real-time solution that addresses all core challenges outlined in the problem statement — sequential cart understanding, cold start, latency constraints, and diversity — while projecting significant business impact.

Key Contributions

- Two-stage retrieval + ranking pipeline with proven scalability architecture
- GRU-based sequential cart encoder capturing meal completion dynamics
- LLM semantic embeddings for cold start handling and food knowledge injection
- Comprehensive A/B testing framework with guardrail metrics protecting user experience
- Projected +10–14% AOV lift and >40% acceptance rate improvement over baseline

Future Scope

- **Multi-objective optimisation:** Balance AOV, diversity, and restaurant fairness simultaneously
- **Contextual bandit online learning:** Adapt recommendations in real-time from live feedback
- **Cross-session personalisation:** Long-term user taste profile with attention-based memory
- **Voice/conversational add-ons:** LLM-powered natural language cart modification
- **Image-based recommendations:** Visual food embeddings from menu item photos

■ All Links & Resources

Resource	Link / Reference
Industry Blogs	https://bytes.swiggy.com/tagged/machine-learning https://doordash.engineering/category/data-science-and-ml/ https://netflixtechblog.com/system-architectures-for-personalization-and-recommendation-e081aa94b5d8 https://www.zomato.com/blog/category/engineering
Google Collab Notebook	https://colab.research.google.com/drive/1_zmiBlsEEXRm5YeZjgbKjrTrOSYM2SjI?usp=sharing
Synthetic Dataset & Research Paper	https://drive.google.com/drive/folders/1TS9zhQtJzzRBCQineO1qSThRUe2mUmqv?usp=sharing
System Design Doc	Included in this PDF — Section 6

Thank You!